Informe: Algoritmos de Caminos Mínimos en Java

Taller 6 – Ciencias de la Computación 2

16 de octubre de 2026

1. Introducción

Los algoritmos de caminos mínimos permiten encontrar las rutas de menor costo entre los vértices de un grafo. Se utilizan en problemas como la planificación de rutas, las redes de comunicación y el análisis de conexiones entre elementos.

En este informe se presentan los algoritmos de Dijkstra, Bellman-Ford y Floyd-Warshall. Para cada uno se explica su funcionamiento paso a paso, sus aplicaciones, su complejidad computacional y un ejemplo de uso basado en el grafo implementado en Java.

2. Algoritmo de Dijkstra

2.1. Funcionamiento paso a paso

Dijkstra calcula los caminos de menor costo desde un vértice de origen hacia los demás vértices de un grafo. Para funcionar correctamente, los pesos de las relaciones deben ser no negativos.

El algoritmo sigue estos pasos:

1. **Inicializar las distancias:** asigna distancia cero al vértice de origen e infinito a todos los demás, porque todavía no se conoce una ruta desde el origen hacia ellos.
2. **Preparar los vértices pendientes:** utiliza una cola de prioridad para organizar los vértices según la menor distancia conocida.
3. **Seleccionar el vértice más cercano:** extrae de la cola el vértice pendiente con la menor distancia conocida.
4. **Examinar sus vecinos:** para cada vecino, suma la distancia del origen al vértice actual y el peso de la relación que los conecta.
5. **Actualizar las distancias:** si el costo calculado es menor que la distancia registrada para el vecino, reemplaza la distancia y registra la nueva ruta. El vecino se incorpora nuevamente a la cola de prioridad para considerar el nuevo valor.
6. **Repetir el proceso:** continúa seleccionando vértices y examinando sus vecinos hasta que la cola quede vacía.
7. **Obtener los resultados:** las distancias registradas corresponden a los caminos mínimos desde el origen hacia los vértices alcanzables. Los vértices que permanecen con distancia infinita no son alcanzables desde el origen.

2.2. Ejemplo de uso

Se utiliza el grafo de ejemplo del código, compuesto por los vértices A, B, C, D, E, F y las siguientes relaciones:

Relación	Conexión	Peso
r1	A – B	4
r2	A – C	2
r3	B – C	1
r4	B – D	5
r5	C – D	8
r6	C – E	10
r7	D – E	2
r8	D – F	6
r9	E – F	3

Al ejecutar Dijkstra desde A, el procedimiento comienza con distancia cero para A e infinito para los demás vértices. Después examina las conexiones y encuentra que llegar a C cuesta 2 y llegar a B por medio de C cuesta 3, mejor que la conexión directa de peso 4.

Continuando el proceso, encuentra una ruta de costo 8 hacia D, de costo 10 hacia E y de costo 13 hacia F.

Los resultados finales son:

Vértice	Distancia desde A	Camino mínimo
A	0	A
B	3	A – C – B
C	2	A – C
D	8	A – C – B – D
E	10	A – C – B – D – E
F	13	A – C – B – D – E – F

2.3. Casos de uso

- Encontrar rutas cortas entre ubicaciones con distancias no negativas.
- Calcular rutas en redes de transporte.
- Determinar caminos de menor costo en redes de comunicación.

2.4. Complejidad computacional

Complejidad temporal:

$$O((V + E) \log V)$$

Esta complejidad corresponde a una implementación con listas de adyacencia y una cola de prioridad basada en un montículo binario.

- Cada vértice puede incorporarse a la cola varias veces cuando se encuentra una distancia mejor. En conjunto, se procesan hasta $O(E)$ incorporaciones relevantes.

- Cada inserción o extracción de la cola de prioridad cuesta $O(\log V)$ en el análisis habitual con una cola que conserva las entradas pendientes y descarta las obsoletas.
- El algoritmo examina las listas de vecinos, cuyo tamaño total es $O(E)$.

Por eso, el costo se expresa como $O((V+E)\log V)$. En grafos conexos suele resumirse como $O(E\log V)$, porque E es al menos del orden de V.

Complejidad espacial:

$$O(V+E)$$

Se necesita espacio para almacenar las listas de adyacencia, que ocupan $O(V+E)$, y las estructuras auxiliares de distancias, vértices anteriores y cola de prioridad, que pueden ocupar hasta $O(V+E)$ en esta implementación. Por tanto, el espacio total mantiene el orden $O(V+E)$.

Dijkstra suele ser eficiente para calcular caminos desde un único origen, siempre que los pesos sean no negativos.

3. Algoritmo de Bellman-Ford

3.1. Funcionamiento paso a paso

Bellman-Ford calcula los caminos mínimos desde un vértice de origen y admite relaciones con pesos negativos. También puede detectar ciclos de peso negativo alcanzables desde el origen.

El algoritmo sigue estos pasos:

1. **Inicializar las distancias:** asigna cero al origen e infinito a los demás vértices.
2. **Obtener las relaciones del grafo:** prepara las conexiones que examinará en cada recorrido. En un grafo no dirigido, cada conexión debe considerarse en ambos sentidos.
3. **Recorrer todas las relaciones:** para cada conexión, comprueba si se puede alcanzar su destino con un costo menor pasando por su origen.
4. **Actualizar los resultados:** cuando encuentra una ruta más económica, cambia la distancia registrada para el vértice de destino y guarda el vértice anterior para reconstruir el camino.
5. **Repetir los recorridos:** examina todas las relaciones hasta completar como máximo $V-1$ recorridos. Esto es suficiente porque, si no hay ciclos negativos relevantes, un camino mínimo puede representarse mediante una ruta simple que contiene como máximo $V-1$ relaciones.
6. **Comprobar ciclos negativos:** realiza un recorrido adicional. Si todavía es posible reducir alguna distancia alcanzable desde el origen, existe un ciclo de peso negativo alcanzable.
7. **Mostrar los resultados:** si no se detecta un ciclo negativo alcanzable, presenta las distancias mínimas y los caminos correspondientes.

3.2. Ejemplo de uso

Se utiliza el mismo grafo de seis vértices y nueve relaciones. Al ejecutar Bellman-Ford desde A, se inicializan las distancias y se recorren las relaciones repetidamente. Como los pesos son positivos, el algoritmo obtiene los mismos resultados que Dijkstra.

Vértice	Distancia desde A	Camino mínimo
A	0	A
B	3	A – C – B
C	2	A – C
D	8	A – C – B – D
E	10	A – C – B – D – E
F	13	A – C – B – D – E – F

En este ejemplo no se detecta ningún ciclo negativo. Si alguna relación tuviera un peso negativo, Bellman-Ford podría seguir calculando caminos mínimos siempre que no existiera un ciclo negativo alcanzable desde A.

3.3. Casos de uso

- Resolver problemas de rutas en grafos con pesos negativos.
- Analizar redes cuyos costos representan ganancias y pérdidas.
- Detectar ciclos de peso negativo alcanzables desde un vértice de origen.

3.4. Complejidad computacional

Complejidad temporal:

$$O(VE)$$

El algoritmo realiza como máximo $V - 1$ recorridos completos de las E relaciones. Cada recorrido examina todas las relaciones, por lo que su costo es $O(E)$. La comprobación adicional de ciclos negativos también recorre las relaciones y no cambia el orden dominante.

En consecuencia, el costo temporal es:

$$O((V - 1)E + E) = O(VE)$$

Complejidad espacial:

$$O(V + E)$$

El algoritmo necesita almacenar el grafo, con un costo de $O(V + E)$, además de las distancias y los vértices anteriores, que ocupan $O(V)$. Por ello, el espacio total es $O(V+E)$.

Bellman-Ford suele ser más lento que Dijkstra, ya que vuelve a examinar todas las relaciones en múltiples recorridos. A cambio, admite pesos negativos y detecta ciclos negativos alcanzables desde el origen.

4. Algoritmo de Floyd-Warshall

4.1. Funcionamiento paso a paso

Floyd-Warshall calcula las distancias mínimas entre todos los pares de vértices de un grafo. A diferencia de los dos algoritmos anteriores, no requiere seleccionar un único vértice de origen.

El algoritmo sigue estos pasos:

1. **Crear la matriz de distancias:** construye una matriz D en la que cada fila y cada columna representan un vértice.
2. **Inicializar la matriz:** asigna cero a la diagonal, porque la distancia de un vértice a sí mismo es cero. Para dos vértices conectados directamente registra el peso de la relación, y para los pares sin conexión directa asigna infinito.
3. **Seleccionar un vértice intermedio:** toma cada vértice k, uno por uno, como posible punto intermedio para mejorar las rutas entre los demás vértices.
4. **Examinar todos los pares:** para cada combinación de origen i y destino j, compara la distancia actual $D[i][j]$ con la ruta que pasa por k, cuyo costo es $D[i][k]+D[k][j]$.
5. **Actualizar la matriz:** si la ruta que pasa por k tiene un costo menor, reemplaza el valor de $D[i][j]$ por ese costo.
6. **Repetir para todos los vértices intermedios:** procesa cada vértice como posible punto intermedio y actualiza las distancias de todos los pares.
7. **Interpretar los resultados:** al finalizar, la matriz contiene las distancias mínimas entre todos los pares de vértices. Una distancia infinita indica que no existe un camino entre ese par.
8. **Detectar ciclos negativos:** si alguna posición de la diagonal tiene un valor negativo, el grafo contiene un ciclo de peso negativo.

4.2. Ejemplo de uso

Al ejecutar Floyd-Warshall sobre el grafo de ejemplo, se calcula la distancia mínima entre cada par de vértices.

	A	B	C	D	E	F
A	0	3	2	8	10	13
B	3	0	1	5	7	10
C	2	1	0	6	8	11
D	8	5	6	0	2	5
E	10	7	8	2	0	3
F	13	10	11	5	3	0

Por ejemplo, la distancia mínima entre A y F es 13, mediante el camino $A \to C \to B \to D \to E \to F$.

La matriz también indica que la distancia mínima entre B y E es 7, mediante el camino $B \to D \to E$.

4.3. Casos de uso

- Consultar las rutas mínimas entre cualquier par de ciudades.
- Analizar redes que requieren numerosas consultas de distancias.
- Obtener una matriz completa de distancias mínimas para una red.

4.4. Complejidad computacional

Complejidad temporal:

$$O(V^3)$$

El algoritmo utiliza tres ciclos anidados: uno para seleccionar el vértice intermedio, otro para recorrer los posibles orígenes y otro para recorrer los posibles destinos. Cada ciclo puede realizar hasta V iteraciones, de modo que el número de comparaciones y actualizaciones es proporcional a:

$$V \times V \times V = V^3$$

Por ello, la complejidad temporal es $O(V^3)$, independientemente de que el grafo tenga pocas o muchas relaciones.

Complejidad espacial:

$$O(V^2)$$

El algoritmo almacena una matriz de distancias con V filas y V columnas. Como cada posición contiene información sobre un par de vértices, la matriz requiere V^2 espacios de almacenamiento. Las variables auxiliares adicionales ocupan espacio constante, por lo que no cambian el orden espacial.

Floyd-Warshall es útil cuando se necesitan distancias entre todos los pares, pero su costo temporal y el espacio de la matriz pueden ser elevados en grafos grandes.

5. Comparación de los algoritmos

Algoritmo	**Complejidad temporal**	**Complejidad espacial**	**Pesos negativos**	**Objetivo principal**
Dijkstra	$O((V + E) \log V)$	$O(V + E)$	No	Caminos desde un origen
Bellman-Ford	$O(VE)$	$O(V + E)$	Sí	Caminos desde un origen y detección de ciclos negativos alcanzables
Floyd-Warshall	$O(V^3)$	$O(V^2)$	Sí, sin ciclos negativos	Caminos entre todos los pares

5.1. Eficiencia y costo computacional

La eficiencia depende del tamaño del grafo y del tipo de consulta que se necesita resolver.

- **Dijkstra:** suele ser la alternativa más eficiente para calcular caminos desde un único origen cuando todos los pesos son no negativos. Su cola de prioridad permite seleccionar los vértices pendientes por distancia, evitando recorrer repetidamente todas las relaciones del grafo.
- **Bellman-Ford:** tiene mayor costo temporal porque examina todas las relaciones en múltiples recorridos. Su principal ventaja es que admite pesos negativos y puede detectar ciclos negativos alcanzables desde el origen.
- **Floyd-Warshall:** realiza un número de operaciones proporcional a V^3 y almacena una matriz de tamaño V^2. Aunque puede ser conveniente para muchas consultas entre distintos pares, su costo crece rápidamente al aumentar la cantidad de vértices.

En términos generales, Dijkstra es adecuado para caminos desde un origen con pesos no negativos; Bellman-Ford, para grafos que pueden contener pesos negativos; y Floyd-Warshall, para calcular las distancias entre todos los pares de vértices.

6. Conclusión

Dijkstra, Bellman-Ford y Floyd-Warshall resuelven problemas de caminos mínimos, pero se diferencian en su funcionamiento, sus restricciones y sus costos computacionales.

Dijkstra suele ser más eficiente cuando los pesos son no negativos y se parte de un único origen. Bellman-Ford necesita más tiempo, pero admite pesos negativos y detecta ciclos negativos alcanzables desde el origen. Floyd-Warshall calcula todas las distancias entre pares, a costa de una complejidad temporal cúbica y un espacio cuadrático.

La selección del algoritmo debe tener en cuenta los pesos de las relaciones, el tamaño del grafo y si se requieren rutas desde un origen o entre todos los pares de vértices.